

Technical Appendix for “Learning Local Feature Masks with Variational Information Bottleneck”

Contents

A1 Detailed Related Work	3
A1.1 Instance-Wise Feature Selection	3
A1.2 Addressing the Co-Adaptation Issue	3
A1.3 Variational Information Bottleneck for Feature Selection	4
A2 Proof for the Lemmas and Theorems in the Main Paper	5
A2.1 Proof of Lemma 1	5
A2.2 Proof of Lemma 2	6
A2.3 Proof of Lemma 3	8
A2.4 Proof of Theorem 1	9
A2.5 Remarks on Independence Assumptions in Lemmas	10
A3 Entropy and Mutual Information Estimation	13
A3.1 Entropy Estimation	13
A3.2 Mutual Information Estimation	14
A3.3 Discussion	15
A4 Derivation of the Regularization Terms in (A26)	15
A5 Optimization and Complexity Analysis	16
A5.1 Optimization Algorithm	16
A5.2 Remarks on mask aggregation in training but majority voting in inference	16
A5.3 Complexity Analysis	17
A6 Complementary Materials for Experiments	17
A6.1 Datasets	17
A6.1.1 Synthetic Datasets	17
A6.1.2 Real-World Datasets	18
A6.1.3 Image Datasets	19
A6.2 Comparison Methods	19
A6.2.1 Training Hyperparameters Summary	20
A6.3 Configuration	21
A6.3.1 Implementation	21
A6.3.2 Training Procedures	21
A6.3.3 Hyperparameter Tuning	22
A6.3.4 Benchmark Methods	22

A6.3.5 Neural Network-Based Methods	23
A6.3.6 The Proposed VIBMask Method	23
A6.4 Evaluation Metrics	25
A6.5 Computing Resources	26
A6.6 Supplementary Synthetic Experiment Results	26
A6.7 Supplementary Real-World Experiment Results	26
A6.7.1 Statistical Significance Analysis	26

A1 Detailed Related Work

A1.1 Instance-Wise Feature Selection

IWFS aims to identify features that are specifically relevant to each individual data instance, enhancing both model generalization and interpretability through context-sensitive explanations. Early work such as TabNet [3] integrated feature selection and prediction into a unified architecture, while more recent approaches adopt a two-module *selector-predictor* design [8, 40, 29], where the selector generates instance-specific masks and the predictor performs classification based on the selected features. A major technical challenge lies in learning sparse, instance-dependent binary masks in a differentiable manner. Existing methods tackle this via various approximations: L2X [8] and gI [26] use a mutual information objective with Gumbel-softmax relaxation [17]; INVASE [41] adopts a reinforcement learning-based actor-critic setup to minimize predictive divergence; LSPIN [40] employs stochastic gates [39]; DIWIFT [23] leverages influence-aware attention; and Copula [32] models feature dependencies via a Gaussian copula. While these methods focus primarily on local mask generation or improved sampling, they often treat feature selection as an isolated task and may struggle in the presence of spurious correlations or heterogeneous input patterns.

Our proposed method, **VIBMask**, reinterprets the instance-wise feature selection (IWFS) problem through the lens of the Variational Information Bottleneck (VIB), jointly optimizing feature informativeness, sparsity, and co-adaptation avoidance in a single principled framework. VIBMask introduces two key innovations: (i) a multi-selector mechanism, where multiple diverse selectors are trained with a diversity-promoting regularizer to capture heterogeneous, context-dependent feature relevance; and (ii) an information-theoretic regularization scheme, which mitigates co-adaptation and redundancy by explicitly controlling the mutual information between features and the target. Together, these components enable VIBMask to produce robust, faithful, and context-aware feature subsets, improving both interpretability and predictive performance compared to prior IWFS and global feature selection methods, across synthetic and real-world datasets.

A1.2 Addressing the Co-Adaptation Issue

A persistent challenge in IWFS is *co-adaptation*, where the selector and predictor become entangled during training such that the selector learns to encode predictive shortcuts, rather than faithfully identifying informative features. This entanglement can result in high predictive accuracy by exploiting spurious correlations, while the selected features remain semantically uninformative and hard to interpret. Several recent methods attempt to mitigate co-adaptation through architectural or training modifications. REAL-X [18] attributes the issue to joint training and introduces an amortized explanation model that learns to approximate the data-generating process independently of the predictor. ProtoGate [19] addresses co-adaptation by structurally decoupling the training of a global-to-local selector from a non-parametric, prototype-based predictor. SUWR [29] treats co-adaptation as a form of information leakage and mitigates it via a greedy, sequential feature selection strategy that limits predictive shortcuts.

In contrast to these architectural and procedural interventions, **VIBMask** approaches co-adaptation from an *information-theoretic* perspective. Rather than modifying the training schedule or predictor structure, VIBMask introduces a regularization term that minimizes the Kullback–Leibler (KL) divergence between the distribution of *unselected* features and the target label. This explicitly discourages the leakage of label-relevant information into discarded features, thereby enforcing that only truly informative features are retained. By directly suppressing mutual information between unselected features and the target, VIBMask provides a principled mechanism for promoting more faithful and interpretable feature selection. Table A1 provides a comparative overview of VIBMask and state-of-the-art IWFS methods, highlighting their respective strategies for addressing co-adaptation and associated design trade-offs.

Method	Year	Venue	Architecture	Unification	Feature Selection Scope			Co-adaptation
				by Eq. (2) of main	Global	Local	Diverse	Mitigation
L2X [8]	2018	ICML	Selector-Predictor	✓	✗	✓	✗	✗
INVASE [41]	2019	ICLR	Selector-Predictor	✗	✗	✓	✗	✗
STG [39]	2020	ICML	Selector-Predictor	✓	✓	✗	✗	✗
gI [26]	2020	NeurIPS	Selector-Predictor	✗	✗	✓	✗	✗
TabNet [3]	2021	AAAI	Encoder-Decoder	✗	✗	✓	✗	✗
REAL-X [18]	2021	AISTATS	Post-hoc Selector	✗	✗	✓	✗	✓
LSPIN [40]	2022	ICML	Selector-Predictor	✓	✗	✓	✗	✗
LLSPIN [40]	2022	ICML	Selector-Predictor	✓	✗	✓	✗	✗
DIWIFT [23]	2023	WWW	Selector-Predictor	✗	✗	✓	✗	✗
Copula [32]	2023	UAI	Selector-Predictor	✗	✗	✓	✗	✗
c-STG [34]	2024	ICML	Selector-Predictor	✓	✗	✓	✗	✗
SUWR [29]	2024	ICML	Selector-Predictor	✗	✗	✓	✗	✓
ProtoGate [19]	2024	ICML	Selector-Predictor	✗	✓	✓	✗	✓
InterpreTabNet [33]	2024	ICML	Encoder-Decoder	✗	✗	✓	✗	✗
VIBMask (Ours)	2025	–	Selector-Predictor	✓	✓	✓	✓	✓

Table A1: **Comparison of Current IWFS Methods.** This table summarizes key characteristics of recent IWFS methods, including their architecture, unification under a common formulation (2), feature selection scope—*global*, *local*, and *diverse*—and whether they mitigate feature co-adaptation. A ✓ indicates the presence of a property. Methods unified by Eq. (2) (e.g., L2X, STG, LSPIN, LLSPIN, c-STG, and VIBMask) share a common theoretical foundation. While most prior methods emphasize local selection, **VIBMask** is the only approach that simultaneously integrates global, local, and diverse feature selection while explicitly addressing co-adaptation. This comprehensive design highlights VIBMask’s contribution as a unified and extensible framework for IWFS.

A1.3 Variational Information Bottleneck for Feature Selection

The VIB framework provides a principled approach to learning compact representations that preserve task-relevant information by minimizing mutual information between input features and latent representations while maximizing their relevance to the target [5, 28]. This property naturally aligns with instance-wise feature selection, where the goal is to isolate predictive features while discarding redundant or spurious ones to enhance interpretability and generalization. Several prior works have applied VIB-inspired techniques to feature selection. For example, Concrete Autoencoder [4] introduces a differentiable selector to identify globally important features, while Gated IB [2] employs a soft gating mechanism and matrix-based Rényi α -mutual information to filter spurious correlations. Drop-Bottleneck [20] adapts the information bottleneck for deterministic compression in reinforcement learning. However, these methods typically focus on learning a *fixed*, globally shared feature mask across all instances, and do not address the *co-adaptation* problem—a central challenge in IWFS where the selector may encode label information directly into the mask, undermining interpretability. Additionally, many rely on Concrete or Gumbel-Softmax relaxations, which are prone to high gradient variance and optimization instability [39]. In the IWFS setting, InterpreTabNet [33] reframes the TabNet architecture as a variational autoencoder, using Gumbel-Softmax-based attention and a variational lower bound to encourage sparsity and diversity.

In contrast, our proposed method, **VIBMask**, explicitly formulates IWFS under the VIB framework with several key innovations. We derive a stable variational lower bound that incorporates a mutual information decorrelation regularizer to disentangle feature relevance from redundancy. VIBMask leverages a continuous reparameterization of binary masks to enable fully differentiable, end-to-end optimization without resorting to high-variance sampling. Unlike prior approaches, VIBMask is designed from the ground up to address the

co-adaptation issue and promote faithful, interpretable feature selection at the instance level.

A2 Proof for the Lemmas and Theorems in the Main Paper

This section provides the proofs and necessary independence assumptions for **Lemmas** 1, 2, 3 and **Theorem** 1 from the main paper. We define **mutual information** $I(X; Y)$, which quantifies the statistical dependence between X and Y , as:

$$I(X; Y) = \mathbb{E}_{X, Y} \left[\log \frac{p(X, Y)}{p(X)p(Y)} \right] = D_{KL}(p(X, Y) || p(X)p(Y)), \quad (\text{A1})$$

This is equivalent to the **Kullback-Leibler (KL) divergence** $D_{KL}(\cdot || \cdot)$ between the joint distribution $p(X, Y)$ and the product of the marginal distributions $p(X)p(Y)$ [35]. Here, $\mathbb{E}_{X, Y}$ denotes the expectation over the joint distribution of X and Y .

Model of masking (local channel). For each coordinate $i \in [d]$, the masked feature is

$$X_{M_i} = \phi_i(X_i, M_i) = \begin{cases} X_i, & M_i = 1, \\ 0, & M_i = 0, \end{cases}$$

i.e., we either keep X_i or replace it by a fixed symbol (here 0). The mask M may be *arbitrarily dependent on* X via the policy $S(M | X)$, possibly with coupling across coordinates. Conditional on M , X_{M_i} depends only on (X_i, M_i) .

No inter-feature independence. The joint distribution of $(X_1, \dots, X_d, Y)$ is arbitrary; in particular, features may be highly correlated (e.g., neighboring pixels in images).

We will use $\pi_i := \Pr(M_i = 1 | X)$ to denote the *local keep probability* of coordinate i under the data-dependent policy. All results remain valid if the masked value 0 is replaced by any constant independent of X and Y .

A2.1 Proof of Lemma 1

Proof. We derive a variational lower bound for the mutual information term $I(X_M; Y)$ in Eq.(2) as follows:

$$\begin{aligned}
I(X_M; Y) &= \int dx_m dy p(x_m, y) \log \frac{p(x_m, y)}{p(x_m)p(y)} \\
&= \int dx_m dy p(x_m)p(y|x_m) \log \frac{p(y|x_m)}{p(y)} \\
&= \int dx_m dy p(x_m)p(y|x_m) \log \frac{p(y|x_m)q(y|x_m)}{p(y)q(y|x_m)} \\
&= \int dx_m dy p(x_m)p(y|x_m) \left[\log \frac{q(y|x_m)}{p(y)} + \log \frac{p(y|x_m)}{q(y|x_m)} \right] \\
&= \int dx_m dy p(x_m)p(y|x_m) \log \frac{q(y|x_m)}{p(y)} + \int dx_m p(x_m) D_{\text{KL}}(p(y|x_m) \| q(y|x_m)) \\
&\geq \int dx_m dy p(x_m)p(y|x_m) \log \frac{q(y|x_m)}{p(y)} \tag{A2}
\end{aligned}$$

$$\begin{aligned}
&= \int dx_m dy p(x_m, y) \log q(y|x_m) - \int dy p(y) \log p(y) \\
&= \mathbb{E}_{p(X_M, Y)} [\log q(Y|X_M)] + H(Y), \tag{A3}
\end{aligned}$$

where $q(y|x_m)$ is an arbitrary variational approximation to the true posterior $p(y|x_m)$, introduced in the third line. Inequality (A2) follows from the non-negativity of the KL divergence.

Finally, note that the entropy term $H(Y)$ is independent of the optimization variable M , and therefore can be omitted when optimizing with respect to M . $\square$

A2.2 Proof of Lemma 2

Proof. For clarity, we present the proof for a single selector (the k -th term on the LHS of the inequality in Lemma 2) and omit the subscript k for notational simplicity. We begin by applying the chain rule of mutual information to decompose the second term $I(X; X_M)$ in Eq. (2):

$$\begin{aligned}
I(X; X_M) &= \sum_{i=1}^d I(X_{\overline{M}_i}; X_1, \dots, X_d \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \\
&= \sum_{i=1}^d \left[I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) + I(X_{\overline{M}_i}; X_{-i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i) \right], \tag{A4}
\end{aligned}$$

where X_{-i} denotes all features except X_i , i.e., $X_{-i} = \{X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_d\}$.

To simplify the mutual information decomposition, we introduce the following formal conditional independence assumptions on the masked features. Let $X_{-i} = \{X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_d\}$ denote all features except X_i , and let $X_{\overline{M}_j}$ denote the j -th masked feature.

Assumption 1 (Conditional independence from other features). *For each feature i , conditioned on the current feature X_i and the previously masked features $X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}$, the masked variable $X_{\overline{M}_i}$ is independent of all other unmasked features X_{-i} :*

$$X_{\overline{M}_i} \perp X_{-i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i. \tag{A5}$$

Assumption 2 (Conditional independence from previous masks). *For each feature i , conditioned on X_i , the masked variable $X_{\overline{M}_i}$ is independent of all previously masked variables:*

$$X_{\overline{M}_i} \perp X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}} \mid X_i. \tag{A6}$$

Discussion. These assumptions formalize the idea that each masked feature $X_{\overline{M}_i}$ depends primarily on its corresponding original feature X_i , and interactions with other features or previous masks are negligible. Assumption 1 ensures that the information between $X_{\overline{M}_i}$ and other features X_{-i} can be ignored, while Assumption 2 guarantees that the dependency between $X_{\overline{M}_i}$ and previous masked variables does not contribute to the mutual information.

While these assumptions greatly simplify the derivation and yield a compact bound, they are strong and may not hold in practice, especially for datasets with correlated features. Later, we introduce a residual term $R(X)$ to account for information leakage when these conditional independence assumptions are violated. The detailed is shown in Lemma A1.

Assumption 1 directly implies that

$$I(X_{\overline{M}_i}; X_{-i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i) = 0. \quad (\text{A7})$$

Assumption 2 allows us to simplify the first term:

$$\begin{aligned} I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) &= H(X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - H(X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i) \\ &= H(X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - H(X_{\overline{M}_i} \mid X_i) \\ &\leq H(X_{\overline{M}_i}) - H(X_{\overline{M}_i} \mid X_i) \\ &= I(X_{\overline{M}_i}; X_i), \end{aligned} \quad (\text{A8})$$

where the second equality follows from Assumption 2, and the inequality follows from the non-negativity of mutual information:

$$I(X_{\overline{M}_i}; X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) = H(X_{\overline{M}_i}) - H(X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \geq 0. \quad (\text{A9})$$

Substituting the simplifications into Eq. (A4), we obtain:

$$\begin{aligned} I(X; X_M) &= \sum_{i=1}^d I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \\ &\leq \sum_{i=1}^d I(X_{\overline{M}_i}; X_i) \\ &= \sum_{i=1}^d [H(X_i) - H(X_i \mid X_{\overline{M}_i})] \\ &= \sum_{i=1}^d [H(X_i) - \pi_i H(X_i \mid X_{\overline{M}_i} = X_i) - (1 - \pi_i) H(X_i \mid X_{\overline{M}_i} = 0)] \\ &= \sum_{i=1}^d [H(X_i) - (1 - \pi_i) H(X_i)] \end{aligned} \quad (\text{A10})$$

$$= \sum_{i=1}^d \pi_i H(X_i). \quad (\text{A11})$$

In Eq. (A10), we use the facts that:

- When $X_{\overline{M}_i} = X_i$, the entropy $H(X_i \mid X_{\overline{M}_i} = X_i) = 0$ since X_i is fully known.
- When $X_{\overline{M}_i} = 0$, no information about X_i is available, hence $H(X_i \mid X_{\overline{M}_i} = 0) = H(X_i)$.

This completes the proof. $\square$

A2.3 Proof of Lemma 3

Proof. For clarity, we present the proof for a single selector (the k -th term on the LHS of the inequality in Lemma 3) and omit the subscript k for notational simplicity. Analogous to the derivation in the proof of Lemma 2, we begin by introducing a moderate independence assumption conditioned on the target variable Y .

Assumption 3 (Conditional independence given the target). *For each feature i , conditioned on the target variable Y , the masked feature $X_{\overline{M}_i}$ is independent of all previously masked features $X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}$:*

$$X_{\overline{M}_i} \perp X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}} \mid Y. \quad (\text{A12})$$

Discussion. This assumption formalizes the idea that, once the target Y is known, each masked feature $X_{\overline{M}_i}$ contributes information about Y independently of the previously masked features. It allows us to upper-bound the mutual information $I(Y; X_{\overline{M}})$ as a sum of individual terms $I(Y; X_{\overline{M}_i})$, greatly simplifying the analysis.

While Assumption 3 makes the derivation tractable, it is a strong assumption that may not hold in real-world datasets where features exhibit correlations even conditional on Y . To account for such dependencies, we can introduce a non-negative residual term $R_Y(X)$ capturing the additional information shared among masked features beyond what is explained by Y , analogous to $R(X)$ in Lemma 2. This is explained in Lemma A2.

Based on Assumption 3, we can derive the following inequality using the same reasoning as in Eq. (A8):

$$I(Y; X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \leq I(Y; X_{\overline{M}_i}). \quad (\text{A13})$$

We now derive an upper bound on $I(Y; X_M)$:

$$\begin{aligned} I(Y; X_M) &= \sum_{i=1}^d I(Y; X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \\ &\leq \sum_{i=1}^d I(Y; X_{\overline{M}_i}) \\ &= \sum_{i=1}^d [H(Y) - H(Y \mid X_{\overline{M}_i})] \\ &= \sum_{i=1}^d [H(Y) - \mathbb{P}(\overline{M}_i = 1)H(Y \mid X_i) - \mathbb{P}(\overline{M}_i = 0)H(Y \mid X_i = 0)] \\ &= \sum_{i=1}^d [H(Y) - (1 - \pi_i)H(Y \mid X_i) - \pi_i H(Y \mid X_i = 0)] \\ &= \sum_{i=1}^d [H(Y) - (1 - \pi_i)H(Y \mid X_i) - \pi_i H(Y)] \\ &= \sum_{i=1}^d (1 - \pi_i) [H(Y) - H(Y \mid X_i)] \\ &= \sum_{i=1}^d (1 - \pi_i) I(X_i; Y). \end{aligned} \quad (\text{A14})$$

The equality in the fourth line follows from the definition $X_{\overline{M}_i} = X_i \odot \overline{M}_i$, which implies: $X_{\overline{M}_i} = X_i$ when $\overline{M}_i = 1$, and $X_{\overline{M}_i} = 0$ when $\overline{M}_i = 0$. The fifth line uses the fact that when X_i is masked (i.e., $M_{c,i} = 0$), it provides no information about Y , so $H(Y \mid X_i = 0) = H(Y)$. Finally, note that the term $\sum_i I(X_i; Y)$ does not depend on the optimization variable and can be treated as a constant during optimization. $\square$

A2.4 Proof of Theorem 1

Proof. Starting from the VIBMask objective for the aggregated mask $M = \frac{1}{K} \sum_{k=1}^K M_k$,

$$\mathcal{J} = I(X_M; Y) - \frac{1}{K} \sum_k \left(\beta I(X_{M_k}; X) + \gamma I(X_{\overline{M}_k}; Y) \right),$$

we apply the three lemmas (Lemmas 1–3) and average over the K selectors. Below we denote the conditional retention probability of feature i by selector k as $\pi_{k,i}(X) = \mathbb{P}(M_{k,i} = 1 \mid X)$.

Lower bound on $I(X_M; Y)$. By Lemma 1 (variational lower bound),

$$I(X_M; Y) \geq \mathbb{E}_{p(Y, X_M)} [\log q(Y \mid X_M)] - \mathbb{E}_{p(Y)} [\log p(Y)].$$

Since $p(X_M, Y) = \mathbb{E}_{X \sim p(X|Y)} [p(X_M \mid X)]$, as shown by

$$\begin{aligned} p(X_M, Y) &= \sum_X p(X, Y, X_M) \\ &= \sum_X p(X, Y) p(X_M \mid X, Y) && \text{(chain rule)} \\ &= \sum_X p(X, Y) p(X_M \mid X) && \text{(by } X_M \perp Y \mid X) \\ &= \mathbb{E}_{X \sim p(X|Y)} [p(X_M \mid X)] && \text{(continuous case via integral),} \end{aligned} \quad (\text{A15})$$

the first term in the bound can equivalently be expressed as the double expectation $\mathbb{E}_{X,Y} \mathbb{E}_{M|X} [\log q(Y \mid X_M)]$.

Upper bound on $I(X_M; X)$. Applying the (pointwise) bound from Lemma 2 to the aggregated mask M (and averaging over the K selectors) yields, for each input X ,

$$\frac{\beta}{K} \sum_{k=1}^K I(X_{M_k}; X) \leq \frac{\beta}{K} \sum_{k=1}^K \sum_{i=1}^d \pi_{k,i}(X) H(X_i),$$

Taking expectation over X gives

$$\frac{\beta}{K} \sum_{k=1}^K I(X_{M_k}; X) \leq \mathbb{E}_X \left[\frac{\beta}{K} \sum_{k=1}^K \sum_{i=1}^d \pi_{k,i}(X) H(X_i) \right].$$

Upper bound on $I(X_{\overline{M}}; Y)$. Applying Lemma 3 (or its residual version) at the selector-average level gives, pointwise in X ,

$$\frac{\gamma}{K} \sum_{k=1}^K I(Y; X_{\overline{M}_k}) \leq \frac{\gamma}{K} \sum_{k=1}^K \sum_{i=1}^d (1 - \pi_{k,i}(X)) I(X_i; Y),$$

Taking expectation over X and multiplying by γ yields

$$\frac{\gamma}{K} \sum_{k=1}^K I(Y; X_{\overline{M}_k}) \leq \mathbb{E}_X \left[\frac{\gamma}{K} \sum_{k=1}^K \sum_{i=1}^d (1 - \pi_{k,i}(X)) I(X_i; Y) \right].$$

Combine the bounds. Combining the three displays and taking the expectation over the data distribution (X, Y) gives

$$\begin{aligned} & I(X_M; Y) - \frac{1}{K} \sum_k \left(\beta I(X_{M_k}; X) + \gamma I(X_{\overline{M}_k}; Y) \right) \\ & \geq \mathbb{E}_{X,Y} \mathbb{E}_{M|X} [\log q(Y | X_M)] - \mathbb{E}_{p(Y)} [\log p(Y)] \\ & \quad - \mathbb{E}_X \left[\frac{\beta}{K} \sum_{k=1}^K \sum_{i=1}^d \pi_{k,i}(X) H(X_i) \right] - \mathbb{E}_X \left[\frac{\gamma}{K} \sum_{k=1}^K \sum_{i=1}^d (1 - \pi_{k,i}(X)) I(X_i; Y) \right]. \end{aligned}$$

Collect the terms that depend on $\pi_{k,i}(X)$. For each feature i and selector k the combined coefficient of $\pi_{k,i}(X)$ is $-\beta H(X_i) + \gamma I(X_i; Y)$. Thus the π -dependent part of the right-hand side equals

$$\mathbb{E}_X \left[\frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d (-\beta H(X_i) + \gamma I(X_i; Y)) \pi_{k,i}(X) \right] = -\mathbb{E}_X \left[\frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d \omega_i \pi_{k,i}(X) \right],$$

where $\omega_i := \beta H(X_i) - \gamma I(X_i; Y)$.

Putting everything together and dropping additive constants that do not depend on the selection probabilities $\{\pi_{k,i}\}$ (specifically $-\mathbb{E}_{p(Y)} [\log p(Y)]$ and $-\frac{\gamma}{K} \sum_{k,i} I(X_i; Y)$), we obtain the variational lower bound

$$\mathcal{L}(q, \pi) = \mathbb{E}_{X,Y} \mathbb{E}_{M|X} \left[\log q(Y | X_M) - \frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d \omega_i \pi_{k,i}(X) \right],$$

This completes the derivation up to the (non-negative) residual terms $R(X)$, $R_Y(X)$ which vanish under the independence assumptions used in the lemmas.

Remark on expectations. The term $\frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d \omega_i \pi_{k,i}(X)$ depends only on X . Therefore, by the law of iterated expectation,

$$\mathbb{E}_X \left[\frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d \omega_i \pi_{k,i}(X) \right] = \mathbb{E}_{X,Y} \mathbb{E}_{M|X} \left[\frac{1}{K} \sum_{k=1}^K \sum_{i=1}^d \omega_i \pi_{k,i}(X) \right].$$

Intuitively, integrating a function of X over the joint distribution (X, Y, M) is equivalent to integrating it over the marginal of X . $\square$

A2.5 Remarks on Independence Assumptions in Lemmas

While the conditional independence assumptions in Lemmas 2 and 3 may be challenging to satisfy under high input correlation, we clarify their theoretical role and practical implications as follows:

- **Theoretical Foundation of Existing Work:** Several prominent IWFS methods—including L2X [8], STG [39], LSPIN [40], and c-STG [34]—are special cases of our framework derived under these assumptions. Their empirical success suggests that practical violations of conditional independence do not inherently degrade selection performance.

- **Information-Theoretic Mapping:** VIBMask provides the first formal link between existing IWFS heuristics and fundamental information-theoretic principles. While a gap between theory and practice persists due to independence violations, narrowing this margin remains a promising direction for future research.
- **Prioritizing Practical Bottlenecks:** In practice, issues such as co-adaptation and single-masking constraints often pose more significant hurdles than independence violations. The superior performance of VIBMask, as detailed in the main results, demonstrates that mitigating these structural bottlenecks yields substantial gains and ensures robustness even when independence assumptions are not strictly met.

Lemma 2 is stated under Assumptions 1–2, which simplify the information decomposition and yield a compact bound. Similarly, Lemma 3 relies on Assumption 3 to upper bound $I(Y; X_{\overline{M}})$ via a chain rule decomposition. In Lemmas A1 and A2, we present generalizations that introduce non-negative residual terms $R(X)$ and $R_Y(X)$, which capture information leakage between features (or masked features conditioned on Y) when these assumptions do not hold; the original bounds are recovered when the residuals vanish. Empirical estimates of $R(X)$ and $R_Y(X)$ on our datasets show that (i) the residuals remain small in our benchmarks, and (ii) the method is effective even under moderate feature dependence.

Lemma A1 (pointwise bound with residual). *For every input X , let $\pi_i(X) = \mathbb{P}(M_i = 1 \mid X)$ denote the conditional selection probability (for one selector; average over k if needed). Define the residual*

$$R(X) := \sum_{i=1}^d I(X_{\overline{M}_i}; X_{-i} \mid X_{\overline{M}_{1:i-1}}, X_i) + \sum_{i=1}^d \left(I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_{1:i-1}}) - I(X_{\overline{M}_i}; X_i) \right)_+,$$

where $(\cdot)_+ := \max\{\cdot, 0\}$. Then

$$I(X; X_M) \leq \sum_{i=1}^d \pi_i(X) H(X_i) + R(X).$$

In particular, when the residual $R(X) = 0$ (e.g., under Assumptions 1–2), the bound reduces to $I(X; X_M) \leq \sum_i \pi_i(X) H(X_i)$.

Proof. We begin by applying the chain rule of mutual information to decompose the second term $I(X; X_M)$ in Eq. (2):

$$\begin{aligned} I(X; X_M) &= \sum_{i=1}^d I(X_{\overline{M}_i}; X_1, \dots, X_d \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \\ &= \sum_{i=1}^d \left[I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) + I(X_{\overline{M}_i}; X_{-i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i) \right], \end{aligned} \quad (\text{A16})$$

where $X_{-i} = \{X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_d\}$.

Rather than setting certain terms to zero, collect the nonnegative cross terms into a residual. Define

$$R_1(X) := \sum_{i=1}^d I(X_{\overline{M}_i}; X_{-i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}, X_i) \geq 0,$$

and note that each term in $R_1(X)$ is nonnegative by nonnegativity of mutual information. For the other summands, write

$$I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \leq I(X_{\overline{M}_i}; X_i) + \left(I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - I(X_{\overline{M}_i}; X_i) \right)_+,$$

and collect the positive excess into

$$R_2(X) := \sum_{i=1}^d \left(I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - I(X_{\overline{M}_i}; X_i) \right)_+ \geq 0.$$

Set the total residual $R(X) := R_1(X) + R_2(X) \geq 0$.

Using these definitions and (A16) we obtain

$$\begin{aligned} I(X; X_M) &= \sum_{i=1}^d I(X_{\overline{M}_i}; X_i \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) + R_1(X) \\ &\leq \sum_{i=1}^d I(X_{\overline{M}_i}; X_i) + R_2(X) + R_1(X) \\ &= \sum_{i=1}^d I(X_{\overline{M}_i}; X_i) + R(X). \end{aligned} \tag{A17}$$

Now proceed as in the original proof to bound each two-point variable $X_{\overline{M}_i}$. For pointwise clarity denote $\pi_i(X) = \mathbb{P}(X_{\overline{M}_i} = X_i \mid X)$ (the conditional selection probability). We have

$$\begin{aligned} I(X_{\overline{M}_i}; X_i) &= H(X_i) - H(X_i \mid X_{\overline{M}_i}) \\ &= H(X_i) - \pi_i(X) H(X_i \mid X_{\overline{M}_i} = X_i) - (1 - \pi_i(X)) H(X_i \mid X_{\overline{M}_i} = 0) \\ &= H(X_i) - (1 - \pi_i(X)) H(X_i), \end{aligned} \tag{A18}$$

where we used that $H(X_i \mid X_{\overline{M}_i} = X_i) = 0$ and $H(X_i \mid X_{\overline{M}_i} = 0) = H(X_i)$. Thus

$$I(X_{\overline{M}_i}; X_i) \leq \pi_i(X) H(X_i).$$

Substituting this into (A17) and summing over i yields the pointwise bound

$$I(X; X_M) \leq \sum_{i=1}^d \pi_i(X) H(X_i) + R(X).$$

This completes the proof. $\square$

Lemma A2 (Conditional residual version of Lemma 3). *Let $\pi_i(X) = \mathbb{P}(M_{c,i} = 1 \mid X)$ denote the conditional selection probability of the i -th feature. Then, for any input X , the mutual information between the target Y and the masked features $X_{\overline{M}}$ satisfies*

$$I(Y; X_{\overline{M}}) \leq \sum_{i=1}^d (1 - \pi_i(X)) I(X_i; Y) + R_Y(X), \tag{A19}$$

where $R_Y(X) \geq 0$ is a residual term capturing the information contribution from dependencies among the masked features conditioned on Y .

Proof. Start with the chain rule of mutual information:

$$I(Y; X_{\overline{M}}) = \sum_{i=1}^d I(Y; X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) \tag{A20}$$

$$= \sum_{i=1}^d I(Y; X_{\overline{M}_i}) + \sum_{i=1}^d \left[I(Y; X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - I(Y; X_{\overline{M}_i}) \right]. \tag{A21}$$

Define the non-negative residual

$$R_Y(X) := \sum_{i=1}^d \left[I(Y; X_{\overline{M}_i} \mid X_{\overline{M}_1}, \dots, X_{\overline{M}_{i-1}}) - I(Y; X_{\overline{M}_i}) \right]_+ \geq 0,$$

which measures the extra information due to dependencies among masked features.

Next, note that $X_{\overline{M}_i} = X_i \odot \overline{M}_i$, so

$$I(Y; X_{\overline{M}_i}) = H(Y) - H(Y \mid X_{\overline{M}_i}) = H(Y) - \pi_i(X)H(Y \mid X_i) - (1 - \pi_i(X))H(Y \mid X_i = 0).$$

Since X_i provides no information about Y when masked ($X_i = 0$), we have $H(Y \mid X_i = 0) = H(Y)$, and thus

$$I(Y; X_{\overline{M}_i}) = (1 - \pi_i(X)) I(X_i; Y).$$

Substituting back, we get the pointwise bound with residual:

$$I(Y; X_{\overline{M}}) \leq \sum_{i=1}^d (1 - \pi_i(X)) I(X_i; Y) + R_Y(X),$$

where $R_Y(X) \geq 0$ captures the extra contribution from dependencies among masked features. This completes the proof. $\square$

A3 Entropy and Mutual Information Estimation

In our variational inference framework, both the entropy $H(X)$ and mutual information $I(X; Y)$ terms are essential components of the variational lower bound. This section details the strategies used to estimate these quantities based on the nature of the variables.

A3.1 Entropy Estimation

The entropy of a variable X reflects the uncertainty of its distribution. Depending on whether X is discrete or continuous, different strategies are used:

Discrete Variable X

When X is categorical, the entropy is estimated using the empirical distribution:

$$H(X) = - \sum_{x \in \mathcal{X}} \hat{p}(x) \log \hat{p}(x), \tag{A22}$$

where $\hat{p}(x)$ is the frequency-based empirical probability of observing value x in the dataset.

Continuous Variable X

For continuous-valued features X , computing $H(X)$ and $I(X; Y)$ is more challenging due to the difficulty in estimating continuous joint and marginal densities. Two common solutions are:

1. **Kernel Density Estimation (KDE):** A non-parametric method that estimates the density of X as:

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N K_h(x - x^{(n)}), \quad (\text{A23})$$

where $K_h(\cdot)$ is a kernel function (e.g., Gaussian) with bandwidth $h > 0$. While conceptually elegant, KDE is computationally expensive and sensitive to the choice of h , especially in high dimensions.

2. **Data Discretization:** To avoid the computational burden of KDE, we adopt a discretization-based approach. Each continuous feature is transformed into categorical values using binning based on its empirical mean μ_X and standard deviation σ_X . For continuous X , we adopt a discretization-based entropy approximation due to the instability and inefficiency of kernel density estimation in high dimensions. We apply a five-bin scheme to map each continuous feature into categorical levels based on its empirical mean μ_X and standard deviation σ_X :

$$X_d = \begin{cases} -2 & \text{if } X < \mu_X - 2\sigma_X, \\ -1 & \text{if } \mu_X - 2\sigma_X \leq X < \mu_X - \sigma_X, \\ 0 & \text{if } \mu_X - \sigma_X \leq X \leq \mu_X + \sigma_X, \\ 1 & \text{if } \mu_X + \sigma_X < X \leq \mu_X + 2\sigma_X, \\ 2 & \text{if } X > \mu_X + 2\sigma_X. \end{cases}$$

This allows us to reuse the discrete entropy estimator with $\hat{p}(x)$ computed over the transformed X_d .

During development, we also implemented density-based estimators such as MINE [6] and kNN-M [25]. Empirically, these sophisticated methods yielded no significant gains in selection performance or training stability over a simpler discretization approach. Consequently, we adopted discretization for the final implementation due to its superior computational efficiency and ease of optimization.

A3.2 Mutual Information Estimation

To estimate $I(X; Y)$ for the variational lower bound, we again distinguish between discrete and continuous feature variables.

Discrete Variable X

For categorical X and (typically discrete) Y , the mutual information is computed via:

$$I(X; Y) \approx \frac{1}{N} \sum_{n=1}^N \log \frac{\hat{p}(x^{(n)}, y^{(n)})}{\hat{p}(x^{(n)})\hat{p}(y^{(n)})}, \quad (\text{A24})$$

where empirical probabilities $\hat{p}(x, y)$, $\hat{p}(x)$, and $\hat{p}(y)$ are estimated from observed samples.

Continuous Variable X

We avoid kernel density estimation due to its computational cost and high sensitivity in high dimensions. Instead, we use the same discretization technique as above to convert X into X_d , and then estimate $I(X_d; Y)$ using the same empirical formula:

$$I(X; Y) \approx I(X_d; Y) = \frac{1}{N} \sum_{i=n}^N \log \frac{\hat{p}(x_d^{(n)}, y^{(n)})}{\hat{p}(x_d^{(n)})\hat{p}(y^{(n)})}. \quad (\text{A25})$$

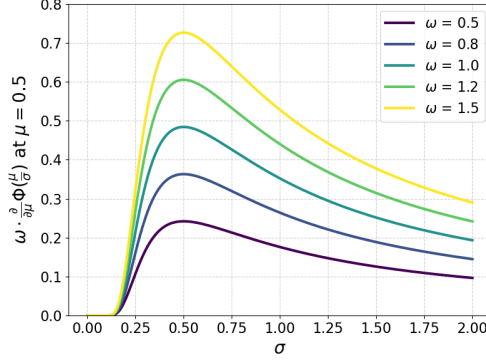


Figure A1: Derivatives of the regularizer w.r.t. μ for $\sigma = [0.001, 2]$. The value of $\omega \frac{\partial}{\partial \mu} \Phi\left(\frac{\mu}{\sigma}\right)|_{\mu=0.5} = \frac{\omega}{\sqrt{2\pi}\sigma^2} e^{-\frac{1}{8\sigma^2}}$ by varying ω from $\{0.5, 0.8, 1.0, 1.2, 1.5\}$.

A3.3 Discussion

These estimators are employed solely to compute entropy and mutual information within the variational lower bound, not for feature selection. Although discretization introduces minor approximation errors, our experiments show that it maintains sufficient estimation fidelity while greatly improving computational efficiency. The original continuous features are preserved for training downstream neural models.

A4 Derivation of the Regularization Terms in (A26)

$$\sum_i \omega_i \pi_{k,i} = \sum_i \omega_i \mathbb{P}(\widetilde{M}_{k,i} > 0) = \sum_i \omega_i \Phi\left(\frac{\mu_{k,i}}{\sigma}\right), \quad (\text{A26})$$

Here we provide the derivation of the regularizer in (A26). The regularization term can be expressed by:

$$\begin{aligned} \sum_{i=1}^d \omega_i p\left(\widetilde{M}_{k,i} > 0\right) &= \sum_{i=1}^d \omega_i p(\mu_{k,i} + \sigma \epsilon_i > 0) \\ &= \sum_{i=1}^d \omega_i (1 - p(\mu_{k,i} + \sigma \epsilon_i \leq 0)) \\ &= \sum_{i=1}^d \omega_i \left(1 - \Phi\left(\frac{-\mu_{k,i}}{\sigma}\right)\right) \\ &= \sum_{i=1}^d \omega_i \Phi\left(\frac{\mu_{k,i}}{\sigma}\right) \\ &= \sum_{i=1}^d \omega_i \left(\frac{1}{2} - \frac{1}{2} \operatorname{erf}\left(-\frac{\mu_{k,i}}{\sqrt{2}\sigma}\right)\right), \end{aligned} \quad (\text{A27})$$

where we use $\widetilde{M}_k$ to denote the local mask generated by the k -th selector S_k .

Following the guideline in [39], we tune the hyperparameter σ . The influence of σ can be analyzed by examining the gradient of the regularizer with respect to μ :

$$\frac{\partial}{\partial \mu_i} \sum_{i=1}^d \omega_i \Phi\left(\frac{\mu_{k,i}}{\sigma}\right) = \omega_i \frac{\partial}{\partial \mu_{k,i}} \Phi\left(\frac{\mu_{k,i}}{\sigma}\right) = \frac{\omega_i}{\sqrt{2\pi}\sigma_i^2} e^{-\frac{\mu_{k,i}^2}{2\sigma_i^2}} \quad (\text{A28})$$

Initially, at the beginning of training ($\mu_{k,i} = 0.5$), these gradients approximate to

$$\frac{\omega_i}{\sqrt{2\pi\sigma_i^2}} e^{-\frac{1}{8\sigma_i^2}}. \quad (\text{A29})$$

For effective sparsification, the gradient, when scaled by the regularization hyperparameter, must exceed the derivative of the loss with respect to μ_i or α_i ; otherwise, $\mu_{k,i}$ will be updated in a counterproductive direction. To promote this, we optimize σ to the value that maximizes the regularization term’s gradient. As shown in Figure A1, this maximum is achieved at $\sigma = 0.5$ for both local masking (with different values of ω from 0.5 to 1.5). Therefore, we maintain $\sigma = 0.5$ throughout our experiments unless explicitly stated otherwise.

A5 Optimization and Complexity Analysis

[t] [1] Dataset $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, number of selectors K , hyperparameters β, γ Trained predictor f_θ and selectors $\{g_{\psi_k}\}_{k=1}^K$

Initialize predictor parameters θ and selector parameters $\{\psi_k\}_{k=1}^K$ (Xavier) Compute feature weights $\omega_i = \beta H(X_i) - \gamma I(X_i; Y)$ for all $i \in \{1, \dots, d\}$, and set noise scale $\sigma \leftarrow \frac{1}{2}$

Bootstrap sampling: $k = 1$ to K Generate bootstrapped dataset $D_k = \{(\mathbf{x}^{(n_k)}, y^{(n_k)})\}_{n_k=1}^N$ by sampling N data points with replacement from the full dataset

not converged **Forward pass:** $k = 1$ to K each $(\mathbf{x}^{(n_k)}, y^{(n_k)}) \in D_k$ Sample noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ Compute logits $\boldsymbol{\mu}_k(\mathbf{x}^{(n_k)}) = g_{\psi_k}(\mathbf{x}^{(n_k)})$ Sample soft mask $\widetilde{\mathbf{M}}_k(\mathbf{x}^{(n_k)}) = \text{clip}(\boldsymbol{\mu}_k(\mathbf{x}^{(n_k)}) + \epsilon, 0, 1)$ Mask input $\widetilde{\mathbf{x}}^{(n_k)} = \mathbf{x}^{(n_k)} \odot \widetilde{\mathbf{M}}_k(\mathbf{x}^{(n_k)})$ Predict $\hat{y}^{(n_k)} = f_\theta(\widetilde{\mathbf{x}}^{(n_k)})$

Objective evaluation: $\hat{R}(\theta, \psi) = \frac{1}{K} \sum_{k=1}^K \left[\frac{1}{|D_k|} \sum_{(\mathbf{x}^{(n_k)}, y^{(n_k)}) \in D_k} L(\hat{y}^{(n_k)}, y^{(n_k)}) + \sum_{i=1}^d \omega_i \Phi\left(\frac{\mu_{k,i}}{\sigma}\right) \right]$

Backpropagation: $k = 1$ to K Update selector parameters $\psi_k \leftarrow \psi_k - \eta \frac{\partial \hat{R}(\theta, \psi)}{\partial \psi_k}$ Update predictor parameters $\theta \leftarrow \theta - \eta \frac{\partial \hat{R}(\theta, \psi)}{\partial \theta}$

A5.1 Optimization Algorithm

The algorithmic procedure for optimizing the proposed VIBMask method is outlined in Algorithm ?? . The proposed **VIBMask** algorithm combines instance-wise feature selection with multiple diverse selectors under a unified variational information bottleneck (VIB) framework. Each selector is trained on a bootstrapped version of the dataset, encouraging diversity and enabling the model to capture heterogeneous, context-dependent feature relevance. For each instance, stochastic masks are generated by perturbing selector logits with Gaussian noise and applied to the input features before passing them through a shared predictor. The overall objective combines prediction loss with an information-theoretic regularization term that mitigates feature redundancy and co-adaptation. This design allows VIBMask to produce robust, interpretable, and context-aware feature subsets across both synthetic and real-world datasets.

A5.2 Remarks on mask aggregation in training but majority voting in inference

In VIBMask, we use averaged (continuous) mask aggregation during training to ensure stable and smooth optimization across multiple selectors. Continuous masks allow gradients to flow effectively through the feature selection process, which improves convergence and prevents instability that could arise from directly using discrete masks. At inference, we apply majority voting to obtain a binary mask for each instance. This

yields interpretable feature selections, which are essential for understanding instance-wise decisions, while preserving the predictive performance learned during training. Our experiments (Tables 1 and 2 of the main paper) show that this combination achieves superior results in both accuracy and interpretability.

Importantly, this approach is consistent with practices in related works such as LSPIN [40] and c-STG [34], where continuous masks are used during training and discrete masks are applied at inference for interpretability. Empirically, we find that the potential performance drift due to this training-inference difference is negligible, and the benefits in stability and interpretability outweigh any minor discrepancies. Thus, maintaining a unified strategy (either always continuous or always discrete) would either compromise stability during training or reduce interpretability at inference, whereas our approach effectively balances both objectives.

A5.3 Complexity Analysis

From a computational perspective, the algorithm scales linearly with the number of samples N , feature dimension d , and number of selectors K . Each selector g_{ψ_k} is implemented as a multi-layer perceptron (MLP) with L_g layers and hidden dimensions $\{h_1, \dots, h_{L_g}\}$, while the predictor f_θ is an MLP with L_f layers and hidden dimensions $\{h'_1, \dots, h'_{L_f}\}$.

The forward pass per selector has complexity

$$O\left(N \sum_{\ell=1}^{L_g} h_{\ell-1} h_\ell\right) \approx O(N \cdot H_g),$$

where H_g represents the total number of operations in the selector MLP, and $h_0 = d$. The forward pass of the predictor for all masked inputs across K selectors has complexity

$$O(K \cdot N \sum_{\ell=1}^{L_f} h'_{\ell-1} h'_\ell) \approx O(K \cdot N \cdot H_f),$$

where H_f is the total cost of the predictor MLP per sample.

Backpropagation roughly doubles the cost of the forward pass. Therefore, the per-iteration gradient computation for selectors is $O(K \cdot N \cdot H_g)$, and for the predictor is $O(K \cdot N \cdot H_f)$. Including T training iterations, the overall per-iteration complexity of VIBMask is

$$O\left(T \cdot K \cdot N \cdot (H_g + H_f)\right).$$

This analysis shows that the cost scales linearly with the number of samples N , selectors K , and the network sizes of the MLPs. Memory usage is dominated by storing the masked inputs and intermediate activations for backpropagation, which scales as $O(K \cdot N \cdot d + K \cdot \sum_{\ell=1}^{L_g} h_\ell + \sum_{\ell=1}^{L_f} h'_\ell)$.

A6 Complementary Materials for Experiments

A6.1 Datasets

A6.1.1 Synthetic Datasets

To ensure fair comparison with existing instance-wise feature selection (IWFS) methods, we adopt the suite of synthetic datasets popularized in this line of work [8, 41, 40, 19, 29]. Each dataset is designed to test a model’s ability to identify complex, instance-specific feature dependencies under controlled conditions in which the ground-truth informative features are known.

Feature distribution and label model. The input features $X \in \mathbb{R}^{11}$ are sampled independently from a standard multivariate Gaussian, $X \sim \mathcal{N}(\mathbf{0}, I_{11})$. The binary label $Y \in \{0, 1\}$ is drawn from a Bernoulli distribution whose class probability is a temperature-scaled sigmoid of a dataset-specific score function $g(X)$:

$$P(Y = 1 \mid X) = \sigma(\alpha \cdot g(X)), \quad \sigma(z) = \frac{1}{1+e^{-z}}, \quad (\text{A30})$$

where the data-temperature $\alpha > 0$ controls how sharply the label depends on $g(X)$. The score functions are:

- **Syn1:** $g(X) = X_1 X_2$ (informative features: $\{1, 2\}$)
- **Syn2:** $g(X) = \sum_{i=3}^6 X_i^2 - 4$ (informative features: $\{3, 4, 5, 6\}$)
- **Syn3:** $g(X) = -10 \sin(0.2 X_7) + |X_8| + X_9 + \exp(-X_{10}) - 2.4$ (informative features: $\{7, 8, 9, 10\}$)
- **Syn4:** $g(X) = g_{\text{Syn1}}(X)$ if $X_{11} < 0$, else $g_{\text{Syn2}}(X)$ (control-flow feature: X_{11})
- **Syn5:** $g(X) = g_{\text{Syn1}}(X)$ if $X_{11} < 0$, else $g_{\text{Syn3}}(X)$ (control-flow feature: X_{11})
- **Syn6:** $g(X) = g_{\text{Syn2}}(X)$ if $X_{11} < 0$, else $g_{\text{Syn3}}(X)$ (control-flow feature: X_{11})

Each dataset contains 5,000 training samples and an independent 5,000-sample test set. Reported numbers are means over five independent seeds (training seed $\in \{0, \dots, 4\}$ and test seed shifted by 100).

Data temperature α . We treat the data temperature α as part of the dataset specification, since it is what determines the difficulty of the classification task once the score function $g(X)$ is fixed. The supplementary specifications of INVASE/L2X correspond to $\alpha = 1$, which is the deterministic-logit setting. Under $\alpha = 1$, the Bayes-optimal accuracies on Syn1 and Syn2/Syn3 are approximately 0.63 and 0.82, respectively—substantially below the 0.81–0.95 band typically reported in the IWFS literature. To match the difficulty regime used by prior work (which we reproduce in Table 1 of the main text), we sharpen α so that the label becomes more nearly deterministic. The per-dataset values used throughout our experiments are:

Dataset	Syn1	Syn2	Syn3	Syn4	Syn5	Syn6
α	5.0	2.0	5.0	3.0	5.0	5.0

We disclose α explicitly so that the benchmark is fully reproducible from Eq. (A30). In our implementation, α is exposed as an argument of the data generator and is held fixed across methods within each dataset; it is not part of the model hyperparameter search.

A6.1.2 Real-World Datasets

We evaluate our proposed method on a collection of ten widely used real-world datasets, which have served as benchmarks in related feature selection and instance-wise methods literature [40, 19]. These datasets span various domains, including text classification, biomedical research, and analysis of image features.

Statistics. For real-world experiment, we use eight datasets that have been widely used in related work [40, 19]. Eight of the datasets are obtained from the scikit-feature dataset repository [21]: **Basehock**, **Relathe**, **PCMAC**, **Colon**, **Tox-171**, **PBMC**, **Lung**, and **Prostate**. The remaining two datasets, **Cnae-9** and **Mfeat-Fourier**, are part of the open-source TabZilla benchmark [27]. A summary of the statistics for eight of these datasets is provided in Table A2. These statistics include the number of samples, features, and classes, as well as the type of data and the predefined train/test split used in our experiments.

Data Preprocessing. Standard data preprocessing techniques were applied consistently across all datasets. For datasets comprising numerical features, we performed feature scaling (e.g., standardization or min-max

Dataset	#Samples	#Features	#Classes	Type	Train / Test
Cnae-9	1080	857	9	Text	280 train / 800 test
Mfeat-Fourier	2000	77	10	Image	400 train / 1500 test
Lung	197	3312	4	Biomedical	100 train / 97 test
Prostate	102	5966	2	Biomedical	70 train / 32 test
Relathe	1355	4322	2	Text	271 train / 1084 test
PCMAC	1845	3289	2	Text	369 train / 1476 test
Colon	62	2000	2	Biomedical	49 train / 13 test
Tox-171	171	5748	4	Biomedical	136 train / 35 test

Table A2: Statistics of used eight real-world datasets from multiple sources

scaling) to normalize the range of values for each feature. For text and image-based datasets (specifically, Mfeat-Fourier which provides extracted image features), the data was utilized in its pre-computed feature vector format. Numerical features within these representations also underwent scaling. Based on the characteristics of these benchmark datasets, no explicit procedures for handling missing values or categorical feature encoding were required.

Data Split. All experiments were conducted using a fixed training and testing data split that was provided with each dataset. The specific number of instances allocated to the training and testing sets for the eight datasets included in Table A2 are detailed therein. Model training was performed exclusively on the designated training sets, and performance evaluation was strictly conducted on the separate, unseen testing sets to ensure an unbiased assessment of generalization capabilities.

A6.1.3 Image Datasets

- **MNIST** [13]: A widely recognized benchmark dataset in machine learning and computer vision, primarily used for image classification tasks. It comprises 70,000 grayscale images of handwritten digits, from 0 to 9, organized into 10 distinct classes. The dataset is standardly split into a training set of 60,000 images and a test set of 10,000 images. Each image has a resolution of 28×28 pixels. Data instances are typically represented as 784-dimensional vectors corresponding to the pixel intensity values, and common preprocessing involves normalizing these values, often scaling them to the range $[0, 1]$ or $[-1, 1]$.
- **Fashion-MNIST** [37]: Designed as a direct replacement for the original MNIST dataset, offering a more challenging benchmark for computer vision algorithms while maintaining the same data format. It consists of 70,000 grayscale images depicting 10 distinct types of clothing items (e.g., t-shirts, trousers, shoes). Similar to MNIST, it is standardly divided into a training set of 60,000 images and a test set of 10,000 images. Each image is a 28×28 pixel grayscale image. Data instances are processed as 784-dimensional vectors of pixel intensities, with preprocessing typically including normalizing the pixel values, similar to the approach used for MNIST.

A6.2 Comparison Methods

In experiments, we compare the proposed VIBMask method with sixteen methods, which can be categorized into three different types: four baseline methods, five global feature selection methods, and eight local (instance-wise) feature selection methods.

- *Baseline.*
Ridge regression [11], KNN [14], SVM [10] and MLP [15]

Method	Selector Arch.	Predictor Arch.	Optimizer	Max Iter.	Early Stop
Ridge	-	-	L-BFGS	10,000	tol= $1e^{-4}$
KNN	-	-	-	-	-
SVM	-	-	-	10,000	tol= $1e^{-3}$
MLP	-	[100, 100, 10]	Adam	5,000	Yes (500)
Lasso	-	-	SAGA	10,000	tol= $1e^{-4}$
RF	-	-	-	-	-
XGBoost	-	-	-	100 rounds	-
STG	[200, 200] or [100, 100]	[100, 100, 10] (tanh)	Adam	7,000	Yes (500)
INVASE	[100, 100]	[200, 200]	Adam	5,000 epochs	-
L2X	[100, 100]	[200, 200]	Adam	7,000	-
LSPIN	[200, 200]	[100, 100, 10] (tanh)	Adam	7,000	Yes (500)
LLSPIN	[200, 200]	Linear stack	Adam	7,000	Yes (500)
ProtoGate	[200, 200] or [100, 100]	-	SGD (NeuralSort $\phi=16.0$)	10,000	Yes (500)
SUWR	-	-	Adam	Default	-
IntTabNet	-	-	Adam	Default	-
REAL-X	[100, 100]	[200, 200]	Adam	1,000	-
VIBMask	[200, 200] or [100, 100]	[200, 200]	AdamW (lr=0.01, wd= $1e^{-4}$)	10,000	Yes (patience=500)

Table A3: Training configurations for VIBMask and the sixteen baseline methods. Each row summarizes the architecture, optimizer, maximum training steps, and early stopping strategy used for each method. A dash (–) indicates the configuration is not applicable.

- *Global feature selection methods.*
 - Linear: Lasso [36]
 - Tree-based methods: Random Forest (RF) [7], and XGBoost [9].
 - Deep methods: STG [39]
- *Local (instancen-wise) feature selection methods.*
 - Selector-Predictor: INVASE [41], L2X [8], LSPIN [40], LLSPIN [40], ProtoGate [19] and SUWR [29].
 - Encoder-Decode: InterpreTabNet (IntTabNet) [33].
 - Post-hoc Selector: REAL-X [18].

To ensure fairness in evaluation, we aligned the architecture of predictor networks across methods where possible, and followed default or recommended configurations from the respective original papers. VIBMask and all neural-based methods were trained using early stopping with validation loss monitoring. Complete scripts and configuration files are available in our codebase.

A6.2.1 Training Hyperparameters Summary

The architectures are reported in terms of hidden layer sizes (e.g., [100, 100] denotes two fully connected layers with 100 neurons each). For ProtoGate, we apply NeuralSort [16] with a temperature $\phi = 16.0$. LLSPIN follows its original implementation using multiple linear layers without activation functions in the predictor. Early stopping refers to terminating training when the validation loss does not improve for a set number of iterations (patience). The dash (–) indicates that the setting is not applicable to the method, such as for non-neural models like Ridge or KNN that do not involve learned selectors or predictors.

A6.3 Configuration

A6.3.1 Implementation

We implemented **VIBMask** using PyTorch, leveraging its modular design to facilitate training, evaluation, and reproducibility. We optimized the implementation with efficient matrix operations in PyTorch [30] to reduce computational overhead, especially when training with multiple selectors in parallel. The mutual information terms were estimated via differentiable approximations using minibatch statistics.

To ensure consistency and fair comparison, we implemented a unified training pipeline for all benchmark methods. Random seeds were fixed across all experiments to ensure deterministic data loading and evaluation. All models were trained and validated on identical splits of the data.

We used open-source implementations for comparison methods whenever possible. For example, we adopted the official codebases for TabNet¹, L2X², INVASE³, REAL-X⁴, LSPIN/LLSPIN⁵, ProtoGate⁶, SUWR⁷ and IntTabNet⁸. Classical baselines like Ridge, KNN, SVM, and Lasso were implemented using `scikit-learn` [31], while XGBoost and Random Forest used their respective official libraries.

The experimental environment, including hardware specifications and library versions, is documented in the accompanying codebase to ensure replicability. All libraries employed in our experiments are licensed under open-source terms: PyTorch under Apache-2.0, `scikit-learn` under BSD-3-Clause, and other implementations under MIT or equivalent permissive licenses.

A6.3.2 Training Procedures

In this section, we describe the key training configurations for **VIBMask** and all benchmark methods. Full implementation details, including data preprocessing, hyperparameter tuning, and random seed control, can be found in the associated code repository. We adopted consistent training protocols across methods to ensure a fair comparison.

- **VIBMask** employs K parallel local selectors and a global predictor. Each local selector is a three-layer MLP with hidden dimension 100 and LeakyReLU [38] activations. The global predictor is a three-layer MLP with hidden sizes [100, 100, 10] and LeakyReLU activations. Training is conducted for 10,000 iterations using the AdamW [24] optimiser with a learning rate selected from the search space and a fixed weight decay of $1e-4$. We apply early stopping based on validation loss with a patience of 500 iterations. The entropy and mutual information penalties are weighted by β and γ , respectively. We performed an ablation study on the batch size and found that a batch size of 64 works well in our experiments. Specifically, we tried batch sizes of 32, 64, 128, and 256. We observed that once the batch size reaches 64, the training process converges both stably and efficiently, and further increasing the batch size does not significantly improve convergence stability.
- **ProtoGate** implements a global-to-local selection mechanism using a three-layer selector network with hidden size 200 (real-world) or 100 (synthetic), and tanh activations in all layers. The predictor is adapted from NeuralSort with temperature $\phi = 16.0$, and a relaxed permutation layer. The model is trained for 10,000 iterations with batch size 64 using SGD and early stopping (patience 500). The feature selector and predictor are jointly trained.

¹<https://github.com/dreamquark-ai/tabnet>

²<https://github.com/Jianbo-Lab/L2X>

³<https://github.com/vanderschaarlab/mlforhealthlabpub/tree/main/alg/invase>

⁴<https://github.com/rajesh-lab/realx>

⁵<https://github.com/jcyang34/lspin>

⁶<https://github.com/SilenceX12138/ProtoGate>

⁷<https://github.com/GarfieldLyu/SUWR>

⁸<https://github.com/jacobyhsi/InterpreTabNet>

- **SUWR** applies a continuous relaxation of feature selection weights and combines both supervised and unsupervised losses to guide selection. It uses a selector network with two hidden layers of size 100 and a predictor MLP with [100, 100, 10] hidden layers using tanh activations. We followed the original implementation and trained SUWR for 7,000 iterations with early stopping (patience 500), using the Adam optimiser with weight decay $1e-4$.
- **InterpreTabNet** uses a two-stage pipeline consisting of a learnable mask generator and a frozen TabNet predictor. The mask generator is trained independently for 5,000 iterations using the Adam optimiser, while the TabNet predictor uses the same architecture as the original paper with a decision embedding width of 8 and feature reuse coefficient of 1.5. The frozen predictor outputs feature importance masks that guide the selector training.
- **STG**, **LSPIN**, and **LLSPIN** use the same selector architecture as **VIBMask**, with predictors being MLPs with [100, 100, 10] layers using tanh activations, except LLSPIN which replaces all activations with linear mappings. The models are trained for 7,000 iterations with early stopping (patience 500). Noise injection standard deviation ρ is set to 0.5.
- **L2X**, **INVASE**, and **REAL-X** follow their published default architectures. The selector has two hidden layers [100, 100] and the predictor [200, 200], both using ReLU activations. L2X is trained for 7,000 iterations, INVASE for 5,000 epochs, and REAL-X for 1,000 iterations.
- **TabNet** uses a decision and attention embedding dimension of 8 and a feature reuse coefficient of 1.5. It is trained using the Adam optimiser with a momentum of 0.3 and gradient clipping at 2.
- **Ridge** is trained for 10,000 iterations to minimise the multinomial loss using the L-BFGS [22] solver, with an early stopping tolerance of $1e-4$.
- **SVM** is trained with an RBF kernel for 10,000 iterations, and a tolerance of $1e-3$.
- **KNN** classifies based on Euclidean distance using uniform weighting over the k nearest neighbours.
- **Lasso** is optimised using the SAGA solver for 10,000 iterations with a tolerance of $1e-4$.
- **Random Forest** is trained with 500 trees, using feature bagging (square root of number of features) and class-balanced weights.
- **XGBoost** is trained using 100 estimators, a learning rate of 1.0, and L_2 regularisation strength of $1e-5$ to minimise the cross-entropy loss.

A6.3.3 Hyperparameter Tuning

To ensure fair and optimal evaluation, we carefully tuned hyperparameters for all compared methods. For each model, we first identified a suitable range of hyperparameters based on existing literature and preliminary experiments to guarantee stable convergence. Within these ranges, we performed a grid search to select the best hyperparameter configuration. The optimal setting was determined by maximizing the balanced accuracy on validation sets, averaged across 25 independent runs.

A6.3.4 Benchmark Methods

For the benchmark methods, we followed a similar hyperparameter tuning strategy as in previous studies [40, 19]. We carefully searched within predefined ranges for both real-world and synthetic datasets to ensure fair comparisons. For tree-based models like XGBoost and Random Forest, we tuned the number of

Method	Sparsity Reg. Strength		Learning Rate	
	Synthetic Data	Real-World Data	Synthetic Data	Real-World Data
STG	{1, 3, 5}	{20, 30, 40, 50}	{1e-1}	{3e-3}
TabNet	{1e-2, 1e-1, 5e-1}	{1e-3, 1e-2, 1e-1, 1}	{1e-2, 2e-2}	{2e-2, 3e-2, 4e-2}
L2X	{1, 5, 10}	{5, 10, 15}	{1e-4}	{1e-3, 5e-4}
INVASE	{1, 1.5, 2}	{1, 2, 2.5}	{1e-4}	{1e-4}
REAL-X	{1, 5, 10, 30, 50}	{10, 20, 30, 50}	{1e-4}	{1e-3}
LSPIN/LLSPIN	{1e-2, 5e-2, 1e-1}	[1e-3, 1.5e-2]	{1e-1}	[5e-2, 1e-1]
ProtoGate	{1e-2, 5e-2, 1e-1}	{5e-4, 1e-3, 1e-2}	{1e-2, 5e-2}	{1e-3, 5e-3}
SUWR	{0.05, 0.1, 0.2}	{0.1, 0.2, 0.3}	{1e-3, 5e-3}	{1e-3}
IntTabNet	{0.1, 0.2, 0.3}	{0.05, 0.1, 0.2}	{1e-3, 5e-3}	{1e-3}

Table A4: Search ranges of key hyperparameters for network-based feature selection methods. Curly braces {} denote grid search, and square brackets [] indicate continuous ranges used with Optuna.

Datasets	Feature Redundancy β	Co-Adaptation γ	Number of Selectors K	Learning Rate φ
Synthetic	{0, 1e-4, 3e-4}	{0, 1e-4, 3e-4}	{3}	{1e-1}
Real-world	{1e-3, 2e-3, 3e-3}	{1e-3, 2e-3, 3e-3, 4e-3, 6e-3}	{1, 2, 3, 4, 5}	{1e-2, 1e-3, 1e-4}

Table A5: Searching ranges of hyperparameters in VIBMask.

estimators, maximum depth, and regularization parameters. For classical models such as Ridge, SVM, KNN, and Lasso, hyperparameters including regularization strength and kernel parameters were optimized using a validation set split from the training data.

For **Ridge regression**, we conducted a grid search over the regularization strength with values in {1, 1e1, 1e2, 1e3}. Similarly, for **SVM**, we searched for the regularization strength within {1e⁻³, 1e⁻², 1e⁻¹, 1, 1e1}, ensuring a balance between underfitting and overfitting.

For **KNN**, we varied the number of nearest neighbors over {1, 3, 5} to determine the optimal choice for different datasets. For **MLP**, we used Optuna [1] to fine-tune the learning rate, searching within the range [1e⁻³, 1e⁻¹] to find the best performing setting.

For **Lasso regression**, we performed a grid search on the regularization strength, exploring the values {1, 1e1, 1e2, 1e3}. For **Random Forest**, we searched for the optimal maximum depth from {3, 5, 7} and the minimum number of instances in a leaf from {2, 3}.

For **XGBoost**, we performed a grid search on the learning rate in {1e⁻², 1e⁻¹} and the maximum depth in {1, 2} to optimize the performance on both synthetic and real-world datasets.

A6.3.5 Neural Network-Based Methods

For most neural network-based methods, including **L2X**, **INVASE**, **SUWR**, and **ProtoGate**, we tuned key hyperparameters such as learning rate, weight decay, temperature, and hidden layer sizes.

Hyperparameter tuning for **LSPIN** and **LLSPIN** was particularly challenging, especially on real-world datasets. To address this, we followed the tuning protocol recommended in the original LSPIN paper [40] and employed the Optuna framework [1] for efficient hyperparameter optimization via Bayesian search. This allowed us to better navigate the complex hyperparameter space of these models and ensure more robust results. We summarize the search ranges of key hyperparameters for network-based feature selection methods in Table A4.

A6.3.6 The Proposed VIBMask Method

Table A5 summarizes the hyperparameter search ranges for **VIBMask** across two types of datasets: **Synthetic** and **Real-world**. The key hyperparameters are:

Dataset	K	Selector hidden	Predictor hidden	Gate	φ	Iters	BS	σ	β	γ	Warm-up	α
Syn1	4	$(128) \times 4$	$(256, 256, 10)$	hard-concrete ($\beta_{\text{HC}}=0.51$)	1.85×10^{-5}	10,000	32	0.89	1×10^{-3}	0	1,000	5.0
Syn2	5	$(128) \times 5$	(128)	hard-concrete ($\beta_{\text{HC}}=0.34$)	2.73×10^{-3}	20,000	128	0.95	0	1×10^{-5}	5,000	2.0
Syn3	3	$(32) \times 5$	$(128, 10)$	gaussian-clip	4.63×10^{-3}	10,000	128	0.37	3×10^{-2}	1×10^{-5}	1,000	5.0
Syn4	5	$(100) \times 2$	$(512) \times 3$	hard-concrete	3.5×10^{-4}	5,000	32	0.71	1×10^{-2}	1×10^{-2}	1,000	3.0
Syn5	3	$(128) \times 4$	$(512) \times 4$	gaussian-clip	1.73×10^{-4}	20,000	32	0.22	1×10^{-2}	1×10^{-1}	1,000	5.0
Syn6	3	$(32) \times 2$	$(256, 256, 10)$	hard-concrete	4.85×10^{-4}	20,000	64	0.63	1×10^{-4}	1×10^{-5}	0	5.0

Table A6: Per-dataset best configurations of **VIBMask** on the synthetic suite, selected by Optuna TPE. K is the number of selectors; *Selector hidden* / *Predictor hidden* list the hidden-layer widths (depth $\times$ width or width tuple); φ is the learning rate; BS is batch size; σ is the gate-noise scale; β and γ are the redundancy and co-adaptation weights in the VIB objective; *Warm-up* is the gate warm-up iteration count; α is the fixed data temperature of Eq. (A30).

- **Local Sparsity (β)**: Controls the strength of feature redundancy. For **Synthetic** data, the range is from 0 to $3e^{-4}$, while for **Real-world** data, it is between $1e^{-3}$ and $6e^{-4}$, allowing more flexibility for complex real-world datasets.
- **Local Sparsity (γ)**: Controls the strength of co-adaptation avoidance. For **Synthetic** data, the range is from 0 to $3e^{-4}$, while for **Real-world** data, it is between $1e^{-3}$ and $6e^{-4}$.
- **Number of Selectors (K)**: This hyperparameter determines how many feature selectors are used. For **Synthetic** datasets, we set $K = 3$, while for **Real-world** datasets, K ranges from 1 to 5 to better handle the complexity of real-world data.
- **Learning Rate (φ)**: The learning rate controls how fast the model learns. For **Synthetic** data, we use a learning rate of $1e^{-1}$, and for **Real-world** data, the range is from $1e^{-2}$ to $1e^{-4}$ to account for more variability in real-world data.

The search ranges for **VIBMask** are designed to balance model performance and computational efficiency. The ranges for **Synthetic** datasets are narrower due to the simpler data structure, while the wider ranges for **Real-world** datasets offer the flexibility needed to capture more complex patterns in the data.

Hyperparameter search protocol. For both synthetic and real-world experiments, all VIBMask hyperparameters are tuned with the Optuna framework [1] using the Tree-structured Parzen Estimator (TPE) multivariate sampler. On the synthetic suite we run 130–180 TPE trials per dataset, optimizing held-out validation accuracy on the training fold; the best configuration is then re-run with five seeds to produce the numbers reported in Table 1 of the main text. The search space spans the number of selectors K , selector and predictor hidden widths, the gating mechanism (hard-concrete vs. Gaussian-clip), learning rate φ , batch size, training iterations, the gate-noise scale σ , the redundancy weight β , the co-adaptation weight γ , and the warm-up length.

Table A6 lists the best per-dataset configurations selected by the search. Two observations are worth flagging. First, the optimal K is dataset-dependent: $K = 3$ suffices on Syn3, Syn5, and Syn6, whereas Syn1, Syn2, and Syn4 favor $K \in \{4, 5\}$, consistent with the heterogeneous-sparsity motivation of multi-selector integration. Second, the co-adaptation weight γ also adapts: it is essentially zero on the unconditional Syn1/Syn2/Syn3 (where label leakage is structurally limited) and grows by several orders of magnitude on the conditional Syn4 (10^{-2}) and Syn5 (10^{-1}), where shortcut routing through X_{11} is most tempting. This pattern is consistent with the discussion in the main text: the decorrelation penalty is most active precisely on the datasets that are most prone to co-adaptation.

A6.4 Evaluation Metrics

In this section, we describe the evaluation metrics used to assess the performance of the models. We evaluate the models in two key areas: *feature discovery (interpretability)* and *label prediction (accuracy)*. The detailed metrics used for evaluation are as follows:

- **Feature Discovery:**

Feature discovery is a critical aspect of the model’s interpretability, focusing on how well the model can identify the most relevant features for the task at hand. To measure this, we use the following metrics:

- **True Positive Rate (TPR):** This metric measures the proportion of relevant features that are correctly identified by the model. It is defined as:

$$\text{TPR} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

where TP represents the number of correctly identified relevant features, and FN represents the number of relevant features that the model failed to identify. A higher TPR indicates better performance in discovering relevant features.

- **False Discovery Rate (FDR):** FDR measures the proportion of irrelevant features that are incorrectly identified as relevant. It is defined as:

$$\text{FDR} = \frac{\text{False Positives (FP)}}{\text{False Positives (FP)} + \text{True Positives (TP)}}$$

where FP represents the number of irrelevant features incorrectly labeled as relevant. A lower FDR is desirable, as it indicates fewer irrelevant features being selected by the model.

- **Control-Flow Selection Rate (CFSR):** CFSR quantifies how often the designated conditional feature (e.g., the eleventh input) is selected, capturing whether the model correctly identifies conditional dependencies. It is defined as:

$$\text{CFSR} = \frac{\text{Number of samples where the conditional feature is selected}}{\text{Total number of samples}}$$

A higher CFSR indicates that the model successfully captures the conditional feature, whereas a low CFSR—especially when coupled with high TPR—suggests that the model may rely on spurious shortcuts rather than explicitly identifying the conditional feature.

- **Label Prediction:**

For evaluating label prediction, we use the following metric to assess the model’s ability to correctly predict the target labels:

- **Accuracy:** Accuracy measures the proportion of correct predictions (both positive and negative) made by the model. It is defined as:

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Predictions (TP + TN + FP + FN)}}$$

where TP represents true positives, TN represents true negatives, FP represents false positives, and FN represents false negatives. Accuracy provides a straightforward assessment of the model’s overall performance in label prediction.

Method	SUWR	VIBMask	ProtoGate	REAL-X	IntTNet	Lasso	STG	MLP	LSPIN	LLSPIN	Ridge	RF	XGB	INVASE	SVM	KNN	L2X
Rank	2.88	3.13	3.38	6.50	7.75	7.88	7.88	8.75	8.75	8.88	10.25	10.88	10.88	12.50	13.13	13.63	16.00

Table A7: Average ranks of all methods across eight datasets (lower is better), computed using the Friedman ranking procedure.

These evaluation metrics comprehensively capture the model’s ability to identify relevant features and predict labels accurately, which are crucial for real-world applications where interpretability and predictive power are both important.

A6.5 Computing Resources

All experimental evaluations were executed on a dedicated workstation. This setup comprised an NVIDIA Tesla V100 GPU, equipped with 32GB of High Bandwidth Memory 2 (HBM2), paired with an Intel® Xeon® Platinum 8168 CPU (2.70 GHz, 24 cores). The underlying operating system was Ubuntu 20.04.5 LTS.

A6.6 Supplementary Synthetic Experiment Results

A6.7 Supplementary Real-World Experiment Results

A6.7.1 Statistical Significance Analysis

To assess whether the observed performance differences among competing methods are statistically significant, we follow the recommendations of Demšar [12] and conduct a Friedman test with a Nemenyi post-hoc analysis on the main results reported in Table 2 of the main paper. All methods are evaluated across $N = 8$ datasets, resulting in a repeated-measures comparison over $k = 17$ methods.

Friedman Test. The Friedman test strongly rejects the null hypothesis that all methods perform equivalently. Specifically, we obtain a Friedman statistic of $\chi^2_F = 69.77$ with a corresponding p-value $p = 1.09 \times 10^{-8}$, which is far below the standard significance threshold $\alpha = 0.05$. This indicates that the performance differences observed across methods are statistically significant, justifying the use of post-hoc pairwise comparisons.

Nemenyi Post-hoc Test. Following the Friedman test, we apply the Nemenyi post-hoc test to identify which pairs of methods differ significantly. Given the number of methods $k = 17$, datasets $N = 8$, and significance level $\alpha = 0.05$, the resulting critical difference (CD) is approximately in the range $CD \approx 3.3$ – 3.6 . Two methods are considered significantly different if the difference in their average ranks exceeds this critical difference.

Discussion of Nemenyi Results. The critical difference analysis reveals several important trends. First, **VIBMask** attains an average rank of 3.13, second only to SUWR (2.88) and just ahead of ProtoGate (3.38); the gap between VIBMask and the next IWFS competitor REAL-X (6.50) is substantially larger (Table A7). The rank difference between VIBMask and lower-ranked methods such as L2X, KNN, SVM, and INVASE exceeds the critical difference, implying that VIBMask is statistically superior to these approaches. Similarly, VIBMask also significantly outperforms most classical baselines and several instance-wise feature selection methods. In contrast, the rank differences between VIBMask and the strongest competing IWFS methods, SUWR and ProtoGate, fall within the critical difference threshold. This suggests that these three methods form a shared top-performing group, with no statistically significant difference among them at $\alpha = 0.05$.

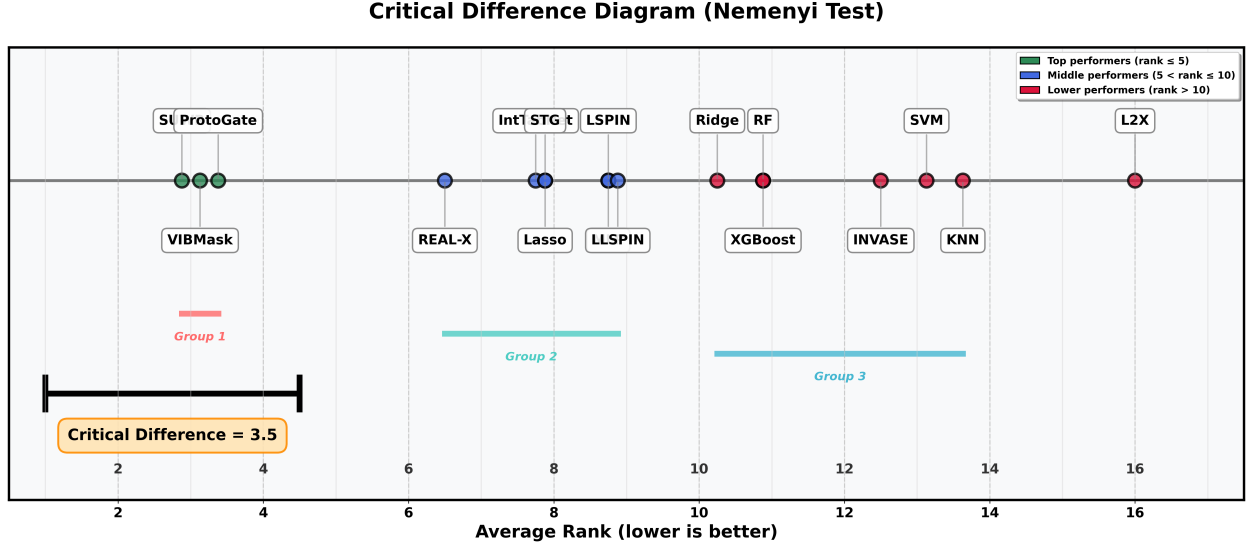


Figure A2: Critical Difference (CD) diagram based on the Nemenyi post-hoc test following the Friedman test across eight datasets. Methods are ordered by increasing average rank (lower is better). Groups of methods connected by a horizontal bar indicate no statistically significant difference at $\alpha = 0.05$. **VIBMask** attains the second-best overall average rank (just behind SUWR) and is statistically superior to most baselines and several IWFS methods, while being statistically indistinguishable from the strongest competitors SUWR and ProtoGate.

Within this group, VIBMask delivers comparable predictive performance while additionally providing a unified information-bottleneck framework with explicit co-adaptation regularization. Figure A2 presents the critical difference (CD) diagram based on the Nemenyi test. Methods are ordered by increasing average rank, and groups of methods connected by a horizontal bar indicate no statistically significant difference at $\alpha = 0.05$. VIBMask is positioned in the leftmost cluster of the diagram, confirming its top-tier status alongside SUWR and ProtoGate.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.
- [2] Francesco Alesiani, Shujian Yu, and Xi Yu. Gated information bottleneck for generalization in sequential environments. *Knowledge and Information Systems*, 65(2):683–705, 2023.
- [3] Sercan Ö Arik and Tomas Pfister. Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6679–6687, 2021.
- [4] Muhammed Fatih Balin, Abubakar Abid, and James Zou. Concrete autoencoders: Differentiable feature selection and reconstruction. In *International conference on machine learning*, pages 444–453. PMLR, 2019.
- [5] David Barber and Felix Agakov. The im algorithm: a variational approach to information maximization. *Advances in neural information processing systems*, 16(320):201, 2004.
- [6] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeswar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and R Devon Hjelm. Mine: Mutual information neural estimation, 2021.

- [7] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [8] Jianbo Chen, Le Song, Martin Wainwright, and Michael Jordan. Learning to explain: An information-theoretic perspective on model interpretation. In *International conference on machine learning*, pages 883–892. PMLR, 2018.
- [9] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [10] Corinna Cortes. Support-vector networks. *Machine Learning*, 1995.
- [11] David R Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 20(2):215–232, 1958.
- [12] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.*, 7:1–30, December 2006.
- [13] Li Deng. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.
- [14] Evelyn Fix. *Discriminatory analysis: nonparametric discrimination, consistency properties*, volume 1. USAF school of Aviation Medicine, 1985.
- [15] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 34:18932–18943, 2021.
- [16] Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations. In *International Conference on Learning Representations*, 2019.
- [17] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations*, 2017.
- [18] Neil Jethani, Mukund Sudarshan, Yindalon Aphinyanaphongs, and Rajesh Ranganath. Have we learned to explain?: How interpretability methods can learn to encode predictions in their interpretations. In *International Conference on Artificial Intelligence and Statistics*, pages 1459–1467. PMLR, 2021.
- [19] Xiangjian Jiang, Andrei Margeloiu, Nikola Simidjievski, and Mateja Jamnik. Protogate: Prototype-based neural networks with global-to-local feature selection for tabular biomedical data. In *Forty-first International Conference on Machine Learning*, 2024.
- [20] Jaekyeom Kim, Minjung Kim, Dongyeon Woo, and Gunhee Kim. Drop-bottleneck: Learning discrete compressed representation for noise-robust exploration. In *International Conference on Learning Representations*, 2021.
- [21] Jundong Li, Kewei Cheng, Suhang Wang, Fred Morstatter, Robert P Trevino, Jiliang Tang, and Huan Liu. Feature selection: A data perspective. *ACM Computing Surveys (CSUR)*, 50(6):94, 2018.
- [22] Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.
- [23] Dugang Liu, Pengxiang Cheng, Hong Zhu, Xing Tang, Yanyu Chen, Xiaoting Wang, Weike Pan, Zhong Ming, and Xiuqiang He. Diwift: Discovering instance-wise influential features for tabular data. In *Proceedings of the ACM Web Conference 2023*, pages 1673–1682, 2023.

- [24] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017.
- [25] Y.P Mack and M Rosenblatt. Multivariate k-nearest neighbor density estimates. *Journal of Multivariate Analysis*, 9(1):1–15, 1979.
- [26] Aria Masoomi, Chieh Wu, Tingting Zhao, Zifeng Wang, Peter Castaldi, and Jennifer Dy. Instance-wise feature grouping. *Advances in Neural Information Processing Systems*, 33:13374–13386, 2020.
- [27] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C., Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [28] Shakir Mohamed and Danilo Jimenez Rezende. Variational information maximisation for intrinsically motivated reinforcement learning. *Advances in neural information processing systems*, 28, 2015.
- [29] Harrie Oosterhuis, Lijun Lyu, and Avishek Anand. Local feature selection without label or feature leakage for interpretable machine learning predictions. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2025.
- [30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [31] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *The Journal of Machine Learning Research*, 12(null):2825–2830, November 2011.
- [32] Hanyu Peng, Guanhua Fang, and Ping Li. Copula for instance-wise feature selection and rank. In *Uncertainty in Artificial Intelligence*, pages 1651–1661. PMLR, 2023.
- [33] Jacob Yoke Hong Si, Wendy Yusi Cheng, Michael Cooper, and Rahul Krishnan. Interpretabnet: Distilling predictive signals from tabular data by salient feature interpretation. In *Forty-first International Conference on Machine Learning*, 2024.
- [34] Ram Dyuthi Sristi, Ofir Lindenbaum, Shira Lifshitz, Maria Lavzin, Jackie Schiller, Gal Mishne, and Hadas Benisty. Contextual feature selection with conditional stochastic gates. In *Forty-first International Conference on Machine Learning*, 2024.
- [35] MTCAJ Thomas and A Thomas Joy. *Elements of information theory*. Wiley-Interscience, 2006.
- [36] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.
- [37] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. <https://github.com/zalandoresearch/fashion-mnist>, 2017. Accessed: 2024-11-22.

- [38] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li. Empirical evaluation of rectified activations in convolutional network, 2015.
- [39] Yutaro Yamada, Ofir Lindenbaum, Sahand Negahban, and Yuval Kluger. Feature selection using stochastic gates. In *International conference on machine learning*, pages 10648–10659. PMLR, 2020.
- [40] Junchen Yang, Ofir Lindenbaum, and Yuval Kluger. Locally sparse neural networks for tabular biomedical data. In *International Conference on Machine Learning*, pages 25123–25153. PMLR, 2022.
- [41] Jinsung Yoon, James Jordon, and Mihaela Van der Schaar. Invase: Instance-wise variable selection using neural networks. In *International conference on learning representations*, 2018.